Project 4: Decision Trees — TA Grading Rubric

Course: EN.601.464 Artificial Intelligence · **Assignment:** Project 4, *Decision Trees: from Entropy to Ensembles*

Scoring: 250 points available, graded out of 200. The 200 base points cover code, voice answers and process. The 50 extra-credit points (Part 8 of the notebook) are a buffer: they can make up for points lost elsewhere or lift a score above 100%. Report the raw total (for example 212/200); the gradebook is set up to accept scores above 200.

Students submit one `submission.zip` built by the notebook's `build_submission()` cell. It contains `project_4_decision_trees.ipynb` and a `recordings/` folder with the audio files, `manifest.json` (durations, timestamps, SHA-256 fingerprints) and `review.html` (one row per question with an inline player). Open `review.html` in a browser to listen; open the notebook to read code and outputs.

Component	Points	How it is graded
Code: 10 coding questions, 10 points each (20 tagged check cells)	100	`instructor/autograde.py` runs the notebook and scores each question; §3.1
Voice answers: 18 required recordings, Q00–Q17, 5 points each	90	§4, graded while listening to `review.html`
Process: docstrings and code comments, function length, Restart & Run All, complete submission	10	§5
Extra credit (Part 8): four coding tasks EC1–EC4 (10 each, autograded) and voice question Q18 (10)	**50**	§3.2 and §4

The notebook is deliberately long (about 17 hours of student work across Parts 0–7, plus an optional Part 8). Expect submissions of 238 cells; the student-facing numbering (Part 0 … Part 9) matches the headings in this rubric.

---

1. What the assignment asks for (grader's map)

Part	Student builds	Deliberate problem the student must explain	Voice questions
0 Setup	runs the recorder, downloads the Kaggle data (253,680 BRFSS respondents, 13.93 % prevalence)	—	Q00 mic check
1 Entropy	`entropy_from_probabilities`, `class_probabilities`, `entropy`	#1 `math domain error` on a pure group (`0·log 0`)	Q01, Q02
2 Information gain	`partition`, `information_gain`, `split_info`, `gain_ratio`	#2 `patient_id` wins information gain (many-values bias)	Q03, Q04
3 ID3	`majority_class`, `best_attribute`, `id3`, `predict_one_naive`, `predict_one`, `classify`	#3 prediction returns `None` for an unseen attribute value	Q05, Q06, Q07
4 Real data & metrics	`confusion_counts`, `sensitivity`, `specificity`, `precision`, `f1_score`, `stratified_folds`	#4 every tree scores 86.07 % (accuracy paradox); duplicates	Q08, Q09, Q10
5 Limits of one tree	`best_threshold`, `should_stop`, `grow_tree` (`entropy_np`, `information_gain_np` and `best_split` are provided)	#5 two halves give different trees (instability); fragmentation of BMI is shown as a "closer look" but has no question	Q11, Q12
6 Ensembles	`bootstrap_indices`, `fit_bagging`, `predict_proba_bagging`, `predict_bagging`, `fit_adaboost`	#6 bagged trees share the same root (correlation)	Q13, Q14, Q15
7 Industry tools	scikit-learn tree, forest, gradient boosting (+ optional XGBoost), `undersample_balanced`, `threshold_for_sensitivity`	#7 precision collapses at real prevalence after balanced training	Q16, Q17 capstone

Part	Student builds	Deliberate problem the student must explain	Voice questions
8 Extra credit (optional)	`best_threshold_fast`, `fit_bagging_oob` + `oob_predict_proba`, `prune_tree`, `threshold_for_cost`	—	Q18 (optional)
9 Submit	`lint_notebook_functions()`, `recording_status()`, `build_submission()`	—	—

2. How to grade voice answers

Listen with `recordings/review.html`. It lists every question with its prompt, the student's recording, its length and timestamp. 1.5× playback speed is fine; the checklists are about content, not delivery.

Each question is scored on a checklist (§4). Award the listed points for each item the student states correctly and in their own words; partial items get half. Do not deduct for accent, hesitation, filler words, or reading from the student's own notes.

General understanding is the goal. Most prompts ask the student to explain an idea or to describe what they saw. A clear explanation in the student's own words earns full credit even if no exact numbers are quoted. A few checklist items are marked **(number)**: those prompts explicitly ask for a value from the student's output ("roughly what entropy did you get", "roughly how many false positives per 1,000"). Only those items require a number, and "roughly" means it: accept anything within the range given in the item. A number that clearly contradicts the student's own notebook output earns nothing for that item.

Using AI to learn is allowed and encouraged (the notebook says so). What is not allowed is outsourcing the recorded explanation itself. Grade what the student says, not how they prepared.

AI conversation logs. Students who used an AI chat tool are asked to save those conversations as PDFs in an `ai_logs/` folder; `build_submission()` packs them into `submission.zip` and prints which logs it found. Read them as context: they show how the student used the tool. The notebook asks for the logs but attaches no points to them, so a missing log or a log that shows a copied function goes to the instructor rather than being penalised by the TA.

Identity. Q00 is a mic check where the student states their name and JHED ID. Use it to confirm that the same voice answers all questions. A clear voice mismatch across questions, or a recording that is obviously synthesized, goes to the instructor (do not grade it; note the question ids).

Script red flags. Reading a prepared answer is fine; reading a generic essay that does not fit the question is not. Treat these as signs that the answer does not engage with the student's work, and cap the question at 50 %: a description of behaviour the student's notebook does not show (e.g., "my tree picked BMI at the root" when the notebook shows `GenHlth`); textbook phrasing with none of the specifics of this notebook; an answer to a different question than the one asked.

Length. Target lengths are guidance for the student. Do not deduct for a short answer that covers the checklist. Recordings under 10 seconds, silent, or unintelligible score 0 for that question.

Re-recordings and external recordings. Only the latest take is submitted. A recording flagged `EXTERNAL` in `review.html` / `manifest.json` was recorded outside the notebook (allowed fallback for microphone problems); grade it normally. A manifest whose SHA-256 does not match the file (`build_submission()` refuses to package such a submission, so this means manual tampering) goes to the instructor.

Q18 is optional. If it is missing, record 0 extra-credit points for it and nothing else; `review.html` marks it "optional".

Integrity escalation. Escalate rather than penalise when: two students' recordings are near-identical in wording; a voice mismatch; a recording that answers a different question than asked in a way suggesting a shared answer bank.

3. Code rubric (100 base points + 40 extra credit)

3.1 Autograded coding questions (100 base points)

Run `python instructor/autograde.py submission.zip`. It executes the notebook (about 5 minutes) and prints, for each of the 10 coding questions, PASS (10 points) or FAIL (0 points) together with the check cell that failed. A question earns its 10 points when every check cell listed for it runs without error. The same checks are visible to students as assert cells, and the notebook lists the 10 questions in its general directions.

If a check fails for a reason unrelated to the question (for example a renamed variable earlier in the notebook), or the student's code is almost right, you may award the partial credit in the last column by hand. Apply the Process deduction in §5 for notebooks that do not run top to bottom.

Question	Check cells (tags)	Functions the student writes	Pts	What the checks verify	Partial credit (manual, out of 10)
C01 Entropy (Part 1)	`entropy_from_probabilities`, `entropy_pure`, `entropy`, `entropy_real`	`entropy_from_probabilities`, `class_probabilities`, `entropy`	10	fair coin = 1 bit, 4 classes = 2 bits, 86/14 = 0.584; `[1.0, 0.0]` → 0.0 without error; class fractions, empty list → 0, lecture labels 0.997; the real label 0.5825 bits	7 if only the pure-group case still crashes; 4 if the formula is right but uses the wrong log base
C02 Information gain and gain ratio (Part 2)	`information_gain`, `gain_ratio`	`partition`, `information_gain`, `split_info`, `gain_ratio`	10	partition sizes and order; gains 0.3845 / 0.2054 / 0.1635 / 0.2054 on the toy data; split info 4.0 (16-way) and 1.0 (8/8); ratios 0.2386 (id) and 0.2434 (BMI); single-value column → 0	6 if information gain passes but gain ratio fails; 3 if partition is right but the gain is not size-weighted
C03 Building the tree with ID3 (Part 3)	`best_attribute`, `id3`	`majority_class`, `best_attribute`, `id3`	10	majority ties → first seen; attribute ties → first candidate; works with `gain_ratio`; reproduces TOY_TREE and the exact LECTURE_TREE; depth 0/1; counts on internal nodes; empty branch inherits the parent majority	7 if the trees are right but counts or the empty-branch case are missing; 3 if only `best_attribute` passes
C04 Predicting with the tree (Part 3)	`predict_naive`, `predict_one`	`predict_one_naive`, `predict_one`, `classify`	10	reproduces training labels on the toy and lecture trees; the naive version still returns `None`; the fallback returns the node majority at the root and at an internal node; lecture error rates 0.0 and 7/15	6 if the fallback uses the global majority instead of the node's; 3 if only the naive walk works
C05 Metrics and stratified folds (Part 4)	`metrics`, `stratified_folds`	`confusion_counts`, `sensitivity`, `specificity`, `precision`, `f1_score`, `stratified_folds`	10	hand-computed counts; zero-division cases return 0; string labels; all-negative predictor on the holdout; 5 disjoint folds covering all indices, sizes within 2, prevalence within 0.5 %	7 if the metrics pass but the folds are random rather than stratified; otherwise 1 per correct metric
C06 Numeric thresholds (Part 5)	`best_threshold`	`best_threshold`	10	perfect split at 3.5; `min_samples_leaf` makes it return `(None, -inf)`; constant column; the dev-data BMI	5 if the midpoints are wrong but the best split is found

Question	Check cells (tags)	Functions the student writes	Pts	What the checks verify	Partial credit (manual, out of 10)
				threshold is optimal by brute force	
C07 The general tree (Part 5)	`grow_tree`	`should_stop`, `grow_tree` (`best_split` is provided)	10	reproduces the Part 3 trees on encoded toy/lecture data; an unseen value falls back and is flagged; real root `HighBP` then `GenHlth <= 2.5` on both sides; `min_samples_leaf=500` respected; categorical mode equals ID3; `choose_pool` size	5 if categorical mode works but numeric splits fail; 3 if only the stopping rules are right
C08 Bagging (Part 6)	`bagging`	`bootstrap_indices`, `fit_bagging`, `predict_proba_bagging`, `predict_bagging`	10	about 63.2 % distinct rows; a list of `Node`s; vote fractions in 1/7 steps; threshold 0 flags everyone	2.5 per working function
C09 AdaBoost (Part 6)	`adaboost`	`fit_adaboost`	10	at least 20 rounds; stumps only; first error = prevalence; the alpha formula; all alphas > 0 and errors in (0, 0.5); error rises in round 2	6 if the weights update correctly but the stop rule is missing; 3 if resampling ignores the weights
C10 Industry tools and the operating point (Part 7)	`sk_tree`, `models`, `undersample`, `threshold`	the scikit-learn tree and `models`, `undersample_balanced`, `threshold_for_sensitivity`	10	entropy criterion, depth 8, all 202,944 training rows, accuracy > 0.86, sensitivity > 0.10; required `models` keys, RF with 100 trees, HGB, AUC RF > 0.80 and HGB > 0.81 ≥ tree; exactly 50 % positives with no duplicates; the largest threshold that reaches 80 % sensitivity	2.5 per passing check cell

Hidden tests (optional): the function signatures are fixed, so staff may add extra asserts (e.g., `entropy` on a 3-class list, `id3` with `score=gain_ratio`, `stratified_folds` with `k=3`). The solution notebook passes all visible checks (100/100).

3.2 Extra-credit code (40 points, autograded)

An unattempted task leaves the stub `return None` in place; its check cell then prints *skipped* and the autograder reports `SKIPPED` (0 points, no penalty). A task only earns points when its check cell prints *checks passed*.

Tag	Function(s)	Pts	What the cell checks	Partial credit (manual)
`ec_fast_threshold`	`best_threshold_fast`	10	identical threshold and gain to `best_threshold` on every numeric column (min leaf 1 and 50), the three hand cases, and faster on BMI	5 if results match but a Python loop over candidates remains; 3 if only the hand cases pass
`ec_oob`	`fit_bagging_oob`, `oob_predict_proba`	10	15 trees with their index arrays; > 99 % of rows covered; rows present in every sample are `nan`; OOB accuracy within 0.03 of the holdout	5 if OOB votes include trees that saw the row
`ec_prune`	`prune_tree`	10	toy tree stays whole when pruned with its own training rows; on dev data leaves drop,	5 if pruning works but is not bottom-up (prunes before

Tag	Function(s)	Pts	What the cell checks	Partial credit (manual)
			validation accuracy never drops, holdout accuracy rises	visiting children)
`ec_cost`	`threshold_for_cost`	10	two hand cases with known answers; on gradient-boosting scores the chosen threshold costs no more than the 0.5 and 80 %-sensitivity thresholds	5 if the right threshold is found but ties or the cost value are wrong

4. Voice question rubric (90 base points + 10 extra credit)

Every required question (Q00–Q17) is worth 5 points; Q18 is extra credit (10 points). For each question: points, target length, the checklist with per-item points, and answers that earn nothing. Items marked **(cause)** are the causal explanation of a deliberate problem and carry the most weight; items marked **(number)** are the only ones that require a value from the student's notebook, and the accepted range is given. Reference values are from the solution notebook; the student's own notebook is the authority if its outputs differ because of a legitimate implementation choice.

Q00 · Mic check · 5 pts · 20–60 s

- 3 — states name and JHED ID clearly (identity reference for the other recordings).
- 2 — gives any prior conception of a decision tree (there is no wrong answer).
- Zero: silence, unintelligible audio, or a different student's voice.

Q01 · What entropy measures · 5 pts · 15–90 s

- 2 — entropy measures uncertainty / how mixed a group is / the expected number of yes-no questions needed to name the label / the average surprisal (the prompt says "think of surprisals": rare outcomes are surprising, $-\log_2 p$, and entropy averages that surprise).
- 2 — 50/50 is 1 bit because one yes/no question resolves it (each outcome has surprisal 1 bit); all-the-same is 0 bits because there is nothing to resolve (the only outcome is certain, surprisal 0).
- 1 **(number)** — mentions the entropy of the real label, roughly 0.5–0.6 bits (reference 0.58), and explains why it is below 1: the label is far from 50/50 (only 14 % positive), so it is mostly predictable.
- Zero: "entropy measures how random the tree is", or no idea what the real value was.

Q02 · Deliberate problem #1, the pure group · 5 pts · 15–30 s

- 1 — describes what happened: a crash (`ValueError: math domain error` or equivalent) on `[1.0, 0.0]`.
- 2 **(cause)** — the formula ends up computing 0 times log(0), and log(0) is undefined.
- 1 — why 0·log 0 = 0 is reasonable: the limit of p log p as p goes to 0 is 0, or in words, a class that never occurs adds no uncertainty.
- 1 — the fix: skip probabilities equal to 0 (or let them contribute 0).
- Zero: "I added a try/except" with no explanation. An epsilon fix earns the fix point only if the student says why 0 is the right value.

Q03 · Information gain in plain words · 5 pts · 45–90 s

- 2 — before vs after: the entropy of the whole group, minus the average entropy of the child groups after the question (the remainder).
- 2 — why size-weighting: the remainder is the *expected* uncertainty after the question; a tiny pure group should not count as much as a large mixed one.
- 1 — `BMI_class` had the highest gain on the toy data, and a sensible reason: its groups are the most separated (normal all healthy, obese mostly diabetic).
- Zero: recites the formula with no idea what it compares, or names the wrong winning attribute.

Q04 · Deliberate problem #2, the identifier · 5 pts · 45–90 s

- 2 **(cause)** — an identifier gives every patient their own branch, so every branch is pure and the gain is as large as it can be (the whole entropy).
- 1 **(cause)** — useless for the next patient: a new id has no branch; the tree learned nothing about the population.
- 1 — the general lesson: information gain favours attributes with many values, because many small groups are easy to make pure.
- 1 — gain ratio divides the gain by the split information (the cost of a many-way split), which penalises identifiers; mentions that it helps only partly, or that the simplest fix is not to feed ids to the learner.
- Zero: "patient_id is correlated with diabetes".

Q05 · ID3 stopping rules · 5 pts · 60–120 s

- 2 — all four stops: pure node; no candidate attributes left; depth limit reached; empty data for a branch value.
- 2 — leaf labels: majority label of the node's records in the first three cases; the **parent's** majority for an empty branch.
- 1 — why the parent's majority: the empty branch has no evidence of its own; the parent summarises every training patient who reached that point; it is the same fallback `predict_one` uses.
- Zero: omits the empty-branch case, or says an empty branch returns `None`/raises.

Q06 · Deliberate problem #3, unseen value · 5 pts · 45–90 s

- 2 **(cause)** — where it got stuck: the root asks `BMI_class`, there is no child for `underweight`, so the walk has nowhere to go and returns `None`.
- 1 — the consequence: `None` cannot be scored (the scoring function refused it); in a clinic this would be a crash.
- 1 — the fix: return the majority class stored at the node where the walk stopped.
- 1 — why sensible: that majority is conditioned on every answer the patient already gave; worse alternatives are crashing, guessing at random, or always saying "no".
- Zero: "I added underweight to the training data".

Q07 · Greedy choice · 5 pts · 30–60 s

- 3 — mechanism: ID3 re-evaluates attributes *within each child*: after the `BMI_class` split, `Age_group` and `PhysActive` are better inside their branches than `HighBP`, so a feature that is second-best globally can be redundant given the first split and never be chosen; the algorithm never looks back.
- 2 — benefit of splitting greedily: it is fast and simple (one pass over the remaining attributes per node), whereas searching every possible tree for the best one is combinatorially infeasible (NP-hard); the greedy tree is usually good enough.
- Half credit on the first item if the student explains only one of the two halves (within-child re-evaluation, or never looking back).
- Zero: "HighBP had low gain".

Q08 · Deliberate problem #4, the 86 % wall · 5 pts · 45–90 s

- 2 **(cause)** — only 14 % have diabetes, so predicting "no" for everyone is right 86 % of the time; depth 0 is exactly that model.
- 1 **(cause)** — a leaf only changes its prediction when more than half of its patients are diabetic; shallow trees never get there, so every leaf still says "no".
- 1 — why accuracy is a poor measure here: it ignores what happens to the minority class, which is the whole point of screening.
- 1 — sensitivity of the shallow trees was 0 (or close to 0: reference 0, 0, 0.05, 0.09 for depths 0–3).
- Zero: "my code was broken", or "86 % is a good model".

Q09 · Sensitivity vs specificity · 5 pts · 45–90 s

- 1 — in their own words: sensitivity = of the people who have the disease, the fraction we flag; specificity = of the healthy, the fraction we correctly leave alone.
- 2 — screening: sensitivity matters more, with a reason (a missed case is a lost diagnosis; a false alarm only costs a confirmatory test).
- 2 — confirmatory test before medication: specificity (or precision) matters more, with a reason (treating a healthy person has side effects).

- Zero: swaps the two definitions.

Q10 · Duplicates and leakage · 5 pts · 30–75 s

- 3 — a row in both folds means the model is tested on a row it memorised, so the estimate is optimistic; yes, that is leakage.
- 2 — what to do: deduplicate before splitting, or keep identical rows in the same fold; may note that identical rows can be different people with identical coarse answers.
- Zero: "duplicates make the model more accurate, which is good".

Q11 · Learning curves · 5 pts · 45–90 s

- 2 — shape: training error keeps falling as depth grows; holdout error falls, bottoms out (around depth 5–8), then rises.
- 1 — the right side is **overfitting**: the deep tree fits the noise of its 50,000 particular rows, not the population.
- 1 — training error keeps falling because a deeper tree can memorise its sample (eventually pure leaves).
- 1 — makes it concrete: the memorising-student picture (perfect on last year's exam, lost on this year's) or the remedy (limit depth or `min_samples_leaf` to stay near the bottom of the holdout curve).
- Zero: calls the right side "underfitting".

Q12 · Deliberate problem #5, instability · 5 pts · 45–90 s

- 1 — the trees differed in their first question (reference: `GenHlth <= 2.5` vs `HighBP`) and disagreed on roughly one patient in seven (reference 14.6 %; accept 10–20 %).
- 2 **(cause)** — the greedy root choice hinges on small differences in gain; once it differs, everything below differs, and a full-depth tree fits the noise of its sample.
- 1 — "high variance" means the predictions depend heavily on which sample the tree happened to see.
- 1 — the physician's unease: rules that change when retrained are hard to trust, audit or explain; the same patient can get a different answer.
- Zero: "different patients, so of course different trees" with no mechanism.

Q13 · Bagging · 5 pts · 45–90 s

- 2 — bagging: draw bootstrap samples (with replacement), grow a tree on each, let them vote; averaging reduces variance.
- 2 — at the 0.5 threshold accuracy went up but sensitivity went down, because a minority-class patient rarely gets a majority of the votes.
- 1 — lowering the threshold raised sensitivity (at a cost in false alarms); the vote fraction is a probability-like score you can threshold, which a single tree's hard labels do not give you.
- Zero: "bagging always improves every metric".

Q14 · Deliberate problem #6, correlated trees · 5 pts · 45–90 s

- 2 **(cause)** — bootstrap samples share most of their rows, so the strongest feature wins the root almost every time; the trees are similar.
- 1 **(cause)** — averaging similar (correlated) trees cancels less noise than averaging independent ones.
- 1 — `max_features` restricts each node to a random subset of columns, forcing trees to ask different questions (decorrelation).
- 1 — what they noticed: the roots spread over many features; ranking quality and the low-threshold scores edged up; the gain was modest because a few features dominate.
- Zero: "max_features makes the trees faster" as the only explanation.

Q15 · AdaBoost · 5 pts · 45–90 s

- 2 — a patient the stump got wrong gets a larger weight (multiplied by e^alpha), a correctly classified one a smaller weight, then weights are renormalised.
- 1 — with uniform weights the first stump predicts "no" for everyone, so its error equals the prevalence (about 0.14).
- 2 — the reweighting makes the diabetic patients count much more, so later stumps must find features that separate them; that is how boosting attends to the minority. (Accept the notebook's sharper version: after one round the

misclassified diabetic patients carry exactly half of the total weight, so "no for everyone" becomes a coin flip.)
- Zero: "boosting trains the trees in parallel".

Q16 · Deliberate problem #7, base rates · 5 pts · 45–90 s

- 1 — sensitivity and specificity belong to the test; precision also depends on how common the disease is (base rate / prevalence).
- 2 **(cause)** — at 14 % prevalence the many healthy people, times the false-positive rate, produce more false alarms than there are true cases to find; on a balanced test set there are far fewer healthy people, so precision looks high.
- 1 **(number)** — roughly 250 false positives per 1,000 screened (reference 252; accept 200–300), or an equivalent statement such as "two false alarms for every true case".
- 1 — conclusion: judge the model at the prevalence it will meet; a balanced test set misleads about precision.
- Zero: "the model overfit the balanced data".

Q17 · Capstone recommendation · 5 pts · 120–180 s

- 1 — a specific recommendation: a model (gradient boosting or random forest expected; a well-argued tree is acceptable) and a threshold, with roughly its sensitivity and specificity (reference for gradient boosting at the 80 % sensitivity point: specificity about 0.70, precision about 0.30).
- 1 — two concrete failure modes, e.g.: self-reported survey features differ from clinic data; 2015 US population vs the deployment population; calibration drift; subgroup performance gaps; unseen or missing values; duplicates inflating validation numbers; false positives overwhelming confirmatory testing.
- 1 — one ethical or practical concern, with a sentence on how to handle it: the Income and Education features the tree leans on (Part 5.8), burden of false positives, missed cases, consent, automation bias.
- 1 — the recommendation is consistent with the student's own results (threshold and metrics match their Part 7 tables within reason).
- 1 — the reflection: one honest sentence contrasting the Q00 answer with what they now understand.
- Zero: a recommendation with no numbers at all, or "86 % accuracy" as the justification.

Q18 · Extra credit: bias and variance · 10 pts · 60–120 s (optional)

- 3 — bias and variance in their own words: bias = systematic error from a model that is too simple or too constrained; variance = error from sensitivity to the particular sample.
- 3 — a variance-reducing experiment from the notebook, correctly explained: bagging or random forests (averaging many trees), `min_samples_leaf`, pruning, or the depth limit.
- 3 — a bias-reducing experiment, correctly explained: AdaBoost (sequentially correcting the stumps' mistakes), deeper trees, or threshold splits replacing coarse categorical ones.
- 1 — a single deep tree suffers mostly from variance; bagging/random forests address that, boosting addresses bias.
- Zero: defines both terms incorrectly or cites no experiment from the notebook.

---

5. Submission checks and penalties (10 points)

The notebook tells students that docstrings and code comments get the most weight here.

Check	Pts	How to verify	Deduction guidance
Docstrings on every function the student wrote, and comments that explain the non-obvious lines	4	`lint_notebook_functions()` output in Part 9 (`has_docstring` column); skim the solution cells for comments	−1 per missing docstring; −1 if the code has no explanatory comments at all; floor 0
Functions about 20 lines or fewer (earnest attempt)	1	same table, `body_lines` column	−1 if any function is over 30 lines; ignore 21–30
Restart & Run All: notebook runs top to bottom without error, cells numbered in order	2	execution counts are 1, 2, 3, … in order; `autograde.py` reports no non-test cell errors	−2 if cells are out of order or any cell errored; −1 if a single cosmetic cell failed (e.g., XGBoost import without try/except). Unattempted extra-credit tasks do not count as errors

Check	Pts	How to verify	Deduction guidance
Complete submission: `submission.zip` built by the notebook, containing the notebook, the 18 required recordings, `manifest.json`, `review.html` and any `ai_logs/*.pdf`; manifest hashes match the files	2	unzip and open `review.html`; `autograde.py` prints the manifest summary	−2 if the zip was assembled by hand with recordings missing or renamed; −1 if 1–2 recordings are missing (and 0 for those questions in §4)
Recordings meet the target length (within reason) and are audible	1	`review.html` shows each duration	−1 if any recording is shorter than half its target or inaudible

Other penalties (apply once each): submitting a PDF or an `.ipynb` outside the zip (−5, and grade from the notebook if recordings were uploaded separately); data files or the kagglehub cache inside the zip (−2); modified test cells (−5 and grade the autograded part by re-running with the original cells; `autograde.py` always uses the course's own `project4_helpers.py`).

---

6. TA workflow and time budget

1. **Batch the autograder.** `for z in submissions/*.zip; do python instructor/autograde.py "$z" --json "${z%.zip}.json"; done` Each run is about 5 minutes on a laptop (the bagging cells dominate); run them in parallel or overnight. Record the base score (out of 100) and the extra-credit score (out of 40) that it prints.
2. **Open the notebook** (the executed copy produced by the autograder is fine) and score §5 (10 pts), plus any manual partial credit from §3.1: about 5 minutes.
3. **Open** `recordings/review.html` and grade §4 top to bottom with the checklists, Q18 included if present. Total audio per student is 15–25 minutes; at 1.5× speed, plus note-taking, budget **20 minutes per student**. Grade one question across several students in a row if you prefer consistency over speed.
4. **Escalate**, do not resolve: voice mismatches, manifest hash mismatches, near-identical recordings across students, and any suspected use of synthesized speech. Note the submission and question ids for the instructor.
5. **Feedback.** Gradescope comments should name the checklist item missed (e.g., "Q08: did not say what a leaf must contain before its prediction flips"). Students see the rubric item names in this document.

Time per submission: about 30 minutes of TA attention plus unattended autograder time. For a section of 100 students that is about 50 TA-hours; plan a grading rota by Part rather than by student if several TAs share the load, so that each Part is graded consistently.

---

7. Reference values from the solution notebook

All experiments are deterministic (seeds fixed; dev sample = 50,000 stratified rows of the 80 % training split; holdout = 50,736 rows). Student values should match these to the printed precision unless their implementation makes a different legitimate choice (e.g., a different tie-break), in which case their notebook output is the reference.

Quantity	Value
Dataset	253,680 × 21 features; prevalence 0.1393; 24,206 exact duplicate rows (35,575 rows, 14.0 %, have a twin)
Entropy of the label	0.5825 bits
Toy data	H = 0.9544; gains: BMI_class 0.3845, HighBP 0.2054, PhysActive 0.2054, Age_group 0.1635; with id: gain 0.9544, gain ratios id 0.2386 vs BMI 0.2434
Part 4 pure-Python ID3 on 50k dev, holdout accuracy by depth 0–6	0.8607, 0.8607, 0.8619, 0.8581, 0.8464, 0.8293, 0.8184 (sensitivity 0, 0, 0.047, 0.092, 0.156, 0.212, 0.245)
Part 5 threshold tree depth-2	root `HighBP`, then `GenHlth <= 2.5` on both sides
Learning curve (min_samples_leaf 5)	holdout error minimum ≈ 0.136 at actual depth 5–8; full depth: train error ≈ 0.09, holdout ≈ 0.17
Fragmentation	categorical: 17,079 leaves, depth 15, unseen-branch rate 8.2 %, holdout acc 0.803, F1 0.280; threshold: 8,719 leaves, depth 38, 0 % unseen, acc 0.799, F1 0.308

Quantity	Value
min_samples_leaf sweep (holdout)	accuracy peaks ≈ 0.865 at m ≈ 100; F1 peaks ≈ 0.31 at m = 5
Instability	roots `GenHlth <= 2.5` vs `HighBP`; disagreement 14.6 %; accuracies 0.832 / 0.828
Bagging, 25 trees, threshold 0.5	acc 0.859, sens 0.174, F1 0.256 (single tree 0.832 / 0.276 / 0.314); at 0.2: sens 0.62, spec 0.80, F1 0.436; roots HighBP 17, GenHlth 8
Random forest, max_features 4	roots spread over ≈ 9 features; AUC 0.788 vs 0.785 for bagging; threshold 0.2: F1 ≈ 0.445, balanced accuracy ≈ 0.70
AdaBoost, 50 stumps	errors 0.139, 0.31, 0.35, 0.42 …; alpha_1 0.91; 50 rounds: acc 0.865, sens 0.19, F1 0.28, AUC 0.82
scikit-learn (full training set)	tree depth 8: acc 0.864, sens 0.136, AUC 0.815; RF 100/msl 20: AUC 0.825; HistGradientBoosting: acc 0.866, AUC 0.828; XGBoost: AUC 0.829
Balanced training (HGB)	real holdout: sens 0.791, spec 0.707, precision 0.304; balanced holdout: precision 0.732; per 1,000 screened: 110 TP, 252 FP, 29 missed
80 % sensitivity operating point	HGB threshold ≈ 0.133, specificity ≈ 0.70, precision ≈ 0.30; RF ≈ 0.69 / 0.30; tree ≈ 0.67 / 0.28
Feature importances (RF)	GenHlth 0.18, BMI 0.18, HighBP 0.15, Age 0.09, HighChol 0.08, DiffWalk 0.07
Extra credit EC1	identical thresholds and gains on all 7 numeric columns; BMI search 44 ms → 3.7 ms (≈ 12× faster)
Extra credit EC2	15 trees; out-of-bag accuracy 0.841 / sensitivity 0.27 vs holdout 0.857 / 0.18 (the test accepts a 0.03 accuracy gap)
Extra credit EC3	unpruned tree on 40k rows: 6,862 leaves, depth 41, holdout acc 0.798; after reduced-error pruning with 10k validation rows: 888 leaves, holdout acc 0.854, 5,479 subtrees collapsed
Extra credit EC4	gradient-boosting scores, FN = 10 × FP: threshold 0.085, sensitivity 0.89, specificity 0.59, precision 0.26, total cost 25,758 (vs 27,132 at the 80 %-sensitivity threshold and 60,608 at 0.5)